

# CodeBLEU Analysis Report: Evaluating Large Language Model Generated Java Code Quality

## Abstract

This study evaluates code similarity between Large Language Model generated Java solutions and reference implementations using CodeBLEU metrics across five programming problems. LLM-generated code achieved an average CodeBLEU score of 0.6542 compared to reference solutions, while maintaining structural integrity despite implementation variations.

## 1. Introduction

CodeBLEU provides a multi-dimensional framework for assessing code similarity by incorporating lexical, syntactic, and semantic components. This research investigates LLM-generated Java code performance through systematic comparison with reference implementations and faulty code samples to quantify similarity patterns and identify divergence areas.

## 2. Methodology

CodeBLEU evaluates code similarity across four dimensions: N-gram Match Score for lexical similarity, Weighted N-gram Match Score with differential weighting, Syntax Match Score for structural similarity, and Dataflow Match Score for semantic similarity. The evaluation encompasses five programming problems including calculator implementation, character counting, caption optimization, collision detection, and prime substring identification.

## 3. Results and Analysis

LLM-generated code achieved an overall average CodeBLEU score of 0.6542 compared to reference implementations, substantially lower than the 0.9849 average for faulty code comparisons. Individual problem performance ranged from 0.4658 for minimum-cost-good-caption to 0.8155 for count-the-number-of-special-characters-ii. The syntax similarity component averaged 0.7945, indicating appropriate structural patterns despite lower overall similarity.

N-gram similarity scores averaged 0.4773, reflecting lexical differences in variable naming and code organization. Syntax and dataflow similarity scores both averaged 0.7945, demonstrating that LLM solutions preserve structural and semantic properties while implementing alternative problem-solving approaches.

## 4. Conclusion

LLM-generated Java code exhibits moderate similarity to reference implementations while maintaining structural integrity. The 0.3307 performance gap reflects implementation strategy differences rather than quality deficiencies. Similarity-based metrics should be complemented with functional correctness assessments for comprehensive code quality evaluation.

## 5. Performance Analysis

Individual problem performance varied significantly. The count-the-number-of-special-characters-ii problem achieved the highest score (0.8155) with strong syntax similarity (0.9286). Robot-collisions scored 0.7682 with good structural match (0.8989). Basic-calculator reached 0.6501 showing adequate syntax structure (0.7673). Sum-of-largest-prime-substrings scored 0.5715 with moderate syntax similarity (0.6808). Minimum-cost-good-caption performed poorest at 0.4658 with significant structural differences (0.6967).

Faulty code maintained very high similarity to reference implementations (0.9849 average), suggesting minimal modifications. LLM code showed moderate similarity (0.6542 average), indicating substantial implementation differences. The performance gap of -0.3307 demonstrates that LLM solutions differ significantly from reference implementations.

LLM code maintains good structural similarity (0.7945 average) indicating proper Java programming constructs. Lower n-gram scores (0.4773 average) suggest different variable names and code organization. The combination indicates LLM implements different but valid algorithmic approaches.

## 6. Implications

LLM-generated code demonstrates structural integrity with proper Java programming patterns. Despite lower similarity scores, the code likely implements correct solutions using alternative approaches. Lower lexical similarity suggests diverse implementations rather than pattern memorization.

The performance gap indicates LLM solutions may not follow conventional algorithmic approaches. Lower n-gram scores suggest differences in coding style and naming conventions. Performance variations across problems indicate inconsistent problem-solving strategies.

High CodeBLEU similarity measures resemblance to reference implementations rather than code quality. LLM code with lower similarity scores may still be functionally correct and efficient. CodeBLEU focuses on structural similarity and may not capture semantic equivalence of different implementations.

Report generated September 22, 2025 using custom CodeBLEU implementation on 5 Java programming problems.